

OPERATING SYSTEMS CONCEPT – LAB

EXP – 08: BANKERS DEADLOCK AVOIDANCE ALGORITHM

```
Activities Terminal Apr 20 17:28 rathnavelu06@localhost:~/rathna/bankeralg — /usr/bin/vim bankers.c

#include <stdio.h>
#define MAX 10
int main()
{
    int n, r, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &r);
    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
    int avail[MAX], finish[MAX]={0}, safe_sequence[MAX];

    // Allocation matrix
    printf("Enter allocation matrix: \n");
    for(i = 0; i < n; i++){
        for(j = 0; j < r; j++){
            scanf("%d", &alloc[i][j]);
        }
    }

    // Max matrix
    printf("Enter Max matrix: \n");
    for(i = 0; i < n; i++){
        for(j = 0; j < r; j++){
            scanf("%d", &max[i][j]);
        }
    }

    // Available resources
    printf("Enter available resource: \n");
    for(i = 0; i < r; i++){
        scanf("%d", &avail[i]);
    }

    // Calculate Need Matrix
    for(i=0; i<n; i++)
        for(j=0; j<r; j++)
            need[i][j]=max[i][j]-alloc[i][j];

    //Print Need Matrix
    printf("\nNeed matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<r; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    int count=0;

    // Banker's algorithm
    while (count < n){
        int safeFound = 0;
        for(i = 0; i < n; i++){
            if(finish[i]==0){
                int canRun = 1;
                for(j = 0; j < r; j++){
                    if(need[i][j] > avail[j])
                    {
                        canRun = 0;
                        break;
                    }
                }
                if(canRun){
                    printf("%d executed, Available now: ", i);
                    for(k = 0; k < r; k++){
                        avail[k] = avail[k] + alloc[i][k];
                        printf("%d ", avail[k]);
                    }
                    printf("\n");
                    safe_sequence[count++] = i;
                    finish[i] = 1;
                    safeFound = 1;
                }
            }
        }
        if(!safeFound){
            printf("System is in unsafe state! Deadlock may occur \n");
            return 0;
        }
    }

    //Print safe sequence
    printf("Safe sequence is: ");
    for(i = 0; i < n; i++){
        printf("%d ", safe_sequence[i]);
    }
    printf("\n");
    return 0;
}
```

```
Activities Terminal Apr 20 17:28
rathnavelu06@localhost:~/rathna/bankergalg$ vi bankers.c
[rathnavelu06@localhost bankergalg]$ gcc bankers.c -o bankers
[rathnavelu06@localhost bankergalg]$ ./bankers
Enter number of processes: 5
Enter number of resource types: 3
Enter allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter available resource:
3 3 2
Need matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1
P1 executed, Available now: 5 3 2
P3 executed, Available now: 7 4 3
P4 executed, Available now: 7 4 5
P0 executed, Available now: 7 5 5
P2 executed, Available now: 10 5 7
Safe sequence is: P1 P3 P4 P0 P2
[rathnavelu06@localhost bankergalg]$
```